

Formal Methods and Specification (LS 2025/26)

Lecture 4:

Correctness of Programs Without Control Structures

Stefan Ratschan

Czech Technical University in Prague



Evropský sociální fond Praha & EU: Investujeme do vaší budoucnosti

I/O Specifications and Program Correctness

Example:

► Input:

- array a , natural number n s.t. $\forall i \in \{0, \dots, n-2\} . a[i] \leq a[i+1]$,
- integer k

► Output: $p \in \{0, \dots, n-1\}$ s.t. $\begin{cases} a[p] = k, & \text{if such a } p \text{ exists, and} \\ -1, & \text{otherwise.} \end{cases}$

A program *satisfies an I/O specification* iff
for every input satisfying the input condition
the program returns an output satisfying the output condition.

This leaves the way how I/O is handled open.

If the specification is clear from the context,
we also say that the *program is correct*.

I/O Specifications and Program Correctness

Example:

► Input:

- array a , natural number n s.t. $\forall i \in \{0, \dots, n-2\} . a[i] \leq a[i+1]$,
- integer k

► Output: $p \in \{0, \dots, n-1\}$ s.t. $\begin{cases} a[p] = k, & \text{if such a } p \text{ exists, and} \\ -1, & \text{otherwise.} \end{cases}$

Example behavior of implementation:

Input: ► $a :$

7	5	2	6	8
---	---	---	---	---

► $k : 6$

Output: -1

Is the implementation **correct**?

I/O Specifications as Assertions

Example:

► Input:

- array a , natural number n s.t. $\forall i \in \{0, \dots, n-2\} . a[i] \leq a[i+1]$,
- integer k

► Output: $p \in \{0, \dots, n-1\}$ s.t. $\begin{cases} a[p] = k, & \text{if such a } p \text{ exists, and} \\ -1, & \text{otherwise.} \end{cases}$

As assertions:

assume $\forall i \in \{0, \dots, n-2\} . a[i] \leq a[i+1]$

...

@ $p \in \{0, \dots, n-1\}$ s.t. $a[p] = k$, if such a p exists, -1 , otherwise

return p

Difference between assertions according to intended usage:

- **assume**: to be ensured by the user, from the outside
- **@**: to be ensured by the given program

Output Requirement as Assertion

```
assume  $\forall i \in \{0, \dots, n-2\} . a[i] \leq a[i+1]$   
... // must not change  $a, k$   
@  $p \in \{0, \dots, n-1\}$  s.t.  $a[p] = k$ , if such a  $p$  exists,  $-1$ , otherwise  
return  $p$ 
```

```
assume  $\forall i \in \{0, \dots, n-2\} . a[i] \leq a[i+1]$   
 $a^* \leftarrow a; k^* \leftarrow k$  // auxiliary variables  $a^*, k^*$   
... // may change  $a, k$   
@  $p \in \{0, \dots, n-1\}$  s.t.  $a^*[p] = k^*$ , if such a  $p$  exists,  $-1$ , otherwise  
return  $p$ 
```

Internal Program Correctness

```
assume I  
@  $i \neq 0$   
 $q \leftarrow 1000/i$ 
```

// input spec

```
    ...  
@  $I < 100$   
 $m \leftarrow a[I]$ 
```

// a has length 100

```
    ...  
@  $O$   
return
```

// output spec

Program Correctness and Assertions

assume $\forall i \in \{0, \dots, n-2\} . a[i] \leq a[i+1]$

...

@ $p \in \{0, \dots, n-1\}$ s.t. $a[p] = k$, if such a p exists, -1 , otherwise

return p

Example behavior of implementation:

Input: ▶ a :

7	5	2	6	8
---	---	---	---	---

 ▶ k : 6

Output: -1

Is the implementation **correct**?

Program Correctness and Assertions

A program execution is *regular* iff
it satisfies all **assume-assertions**.

A program is (*partially*) *correct* iff
every regular program execution
from the first program line to a program line with an @-assertion,
satisfies this **@-assertion**.

Definitions not formal: “program execution” undefined

Tiny imperative programming language,
for which we all agree on its behavior

Definition of Program Correctness: Variants

I/O specification, no assertions:

correctness: program satisfies specification

program with I/O specification (+ internal correctness) encoded as assertions:

correctness via assertion satisfaction

partial vs. total correctness

rest of this lecture:

How to prove correctness of programs without control structure?

Program Correctness: Assignments

$y \leftarrow x^2$

$z \leftarrow x + y$ correct?

@ $z \geq 0$

$$\forall x, y, z . [y = x^2 \wedge z = x + y] \Rightarrow z \geq 0$$

demo (assignments.cvc)

Reminder: a formula is valid iff its universal closure is valid:

For a formula ϕ with free variables v

$$\models \phi \text{ iff } \models \forall v . \phi$$

Hence it suffices to prove

$$[y = x \wedge z = x + y] \Rightarrow z \geq 0$$

Program Correctness: Assumptions

assume $x \geq 5$

$y \leftarrow x$

$z \leftarrow x + y$

@ $z \geq 0$

// changed line!

$$[x \geq 5 \wedge y = x \wedge z = x + y] \Rightarrow z \geq 0$$

demo (assignments_assume.cvc)

Program Correctness: Assumptions

assume $x \geq 5$

$y \leftarrow x$

$x \leftarrow -10$

$z \leftarrow x + y$

@ $z \geq 0$

x	y	z
5		
5	5	
-10	5	
-10	5	-5

$$[x \geq 5 \wedge y = x \wedge x = -10 \wedge z = x + y] \Rightarrow z \geq 0$$

demo (assignments_assume_overwrite.cvc)

Shows program to be correct, but it is not! Where is the problem?

Program Correctness: Assumptions

assume $x \geq 5$

$y \leftarrow x$

$x_1 \leftarrow -10$

$z \leftarrow x_1 + y$

@ $z \geq 0$

x	y	z
5		
5	5	
-10	5	
-10	5	-5

$$[x \geq 5 \wedge y = x \wedge x_1 = -10 \wedge z = x_1 + y] \Rightarrow z \geq 0$$

demo (assignments_renamed.cvc) counter-example

$\{x \mapsto 5, y \mapsto 5, x_1 \mapsto -10, z \mapsto -5\}$

(satisfies left-hand side, but not right-hand side of implication)

static single assignment form (SSA)

widely used in compiler construction, see Wikipedia

Program Correctness: Assignments

Original program:

$x \leftarrow 1$

$x \leftarrow x + 1$

@ $x \geq 10$

SSA:

$x \leftarrow 1$

$x_1 \leftarrow x + 1$

@ $x_1 \geq 10$

$$[x = 1 \wedge x_1 = x + 1] \Rightarrow x_1 \geq 10$$

Programmers do this in their head, names

- ▶ “the value of variable x in the first line”
- ▶ “the value of variable x from line two on”
- ▶ ...

Program Correctness: Array Assignments

$a[i] \leftarrow 7$
 $a[i] \leftarrow 5$
...

SSA???

Function symbols of arrays:

- ▶ $\cdot[\cdot] : \mathcal{A}[\mathcal{T}] \times \mathcal{N} \rightarrow \mathcal{T}$
- ▶ $\text{write} : \mathcal{A}[\mathcal{T}] \times \mathcal{N} \times \mathcal{T} \rightarrow \mathcal{A}[\mathcal{T}]$

Using those function symbols:

$a \leftarrow \text{write}(a, i, 7)$
 $a \leftarrow \text{write}(a, i, 5)$
...

SSA

$a_1 \leftarrow \text{write}(a, i, 7)$
 $a_2 \leftarrow \text{write}(a_1, i, 5)$
...

Program Correctness: User Input

assume $x \geq 5$

input x

$z \leftarrow x + y^2$

@ $z \geq 0$

$$[x \geq 5 \wedge z = x + y^2] \Rightarrow z \geq 0$$

???

Program Correctness: User Input

Original program:

assume $x \geq 5$

input x

$z \leftarrow x + y^2$

@ $z \geq 0$

SSA:

assume $x \geq 5$

input x_1

$z \leftarrow x_1 + y^2$

@ $z \geq 0$

$$[x \geq 5 \wedge z = x_1 + y^2] \Rightarrow z \geq 0$$

Correctness of Programs without Control Structures

Basic path: program of the form

c_1

$\dots$

c_n

@ α

where

- ▶ $c_1, \dots, c_n$ neither contains control structures nor @
- ▶ α is a logical formula,

Here: assignments, **assume**, **input**

Static Single Assignment Form

Whenever a variable is assigned a new value (e.g., **input**),
rename variable to new one:

Example:

```
assume  $x \geq 0$   
input  $x$   
assume  $2x - 1 \geq 3$   
 $x \leftarrow x - 2$   
assume  $x \geq 0$   
input  $x$   
assume  $\neg 2x - 1 \geq 3$   
 $x \leftarrow x - 1$   
 $@ x \neq 0$ 
```

```
assume  $x \geq 0$   
input  $x_1$   
assume  $2x_1 - 1 \geq 3$   
 $x_2 \leftarrow x_1 - 2$   
assume  $x_2 \geq 0$   
input  $x_3$   
assume  $\neg 2x_3 - 1 \geq 3$   
 $x_4 \leftarrow x_3 - 1$   
 $@ x_4 \neq 0$ 
```

Notation: SSA(P)

Static Single Assignment Form

$$SSA_V(@ \alpha) := @ \alpha$$

$$SSA_V(\phi; P) := \phi; SSA_V(P)$$

for all commands ϕ that do not change variable values

$$SSA_V(\mathbf{input} \ v; P) := \mathbf{input} \ v_{new}; SSA_{V \cup \{v_{new}\}}(P[v \leftarrow v_{new}])$$

where v_{new} is a variable s.t. $v_{new} \notin V$.

$$SSA_V(v \leftarrow t; P) := v_{new} \leftarrow t; SSA_{V \cup \{v_{new}\}}(P[v \leftarrow v_{new}])$$

where v_{new} is a variable s.t. $v_{new} \notin V$.

$$SSA(P) := SSA_{vars(P)}(P)$$

Example:

$$\begin{aligned}
 & SSA \left(\begin{array}{l} \text{assume } x \geq 0 \\ \text{input } x \\ \text{assume } 2x - 1 \geq 3 \\ x \leftarrow x - 2 \\ \text{assume } x \geq 0 \\ \text{input } x \\ \text{assume } \neg 2x - 1 \geq 3 \\ x \leftarrow x - 1 \\ @ x \neq 0 \end{array} \right) = SSA_{\{x\}} \left(\begin{array}{l} \text{assume } x \geq 0 \\ \text{input } x \\ \text{assume } 2x - 1 \geq 3 \\ x \leftarrow x - 2 \\ \text{assume } x \geq 0 \\ \text{input } x \\ \text{assume } \neg 2x - 1 \geq 3 \\ x \leftarrow x - 1 \\ @ x \neq 0 \end{array} \right) = \\
 & \left(\text{assume } x \geq 0 \right); SSA_{\{x\}} \left(\begin{array}{l} \text{input } x \\ \text{assume } 2x - 1 \geq 3 \\ x \leftarrow x - 2 \\ \text{assume } x \geq 0 \\ \text{input } x \\ \text{assume } \neg 2x - 1 \geq 3 \\ x \leftarrow x - 1 \\ @ x \neq 0 \end{array} \right) =
 \end{aligned}$$

Example:

$$\begin{aligned}
 & \left(\begin{array}{l} \text{assume } x \geq 0 \\ \text{input } x_1 \end{array} \right); SSA_{\{x, x_1\}} \left(\begin{array}{l} \text{assume } 2x_1 - 1 \geq 3 \\ x_1 \leftarrow x_1 - 2 \\ \text{assume } x_1 \geq 0 \\ \text{input } x_1 \\ \text{assume } \neg 2x_1 - 1 \geq 3 \\ x_1 \leftarrow x_1 - 1 \\ @ x_1 \neq 0 \end{array} \right) = \\
 & \left(\begin{array}{l} \text{assume } x \geq 0 \\ \text{input } x_1 \\ \text{assume } 2x_1 - 1 \geq 3 \end{array} \right); SSA_{\{x, x_1\}} \left(\begin{array}{l} x_1 \leftarrow x_1 - 2 \\ \text{assume } x_1 \geq 0 \\ \text{input } x_1 \\ \text{assume } \neg 2x_1 - 1 \geq 3 \\ x_1 \leftarrow x_1 - 1 \\ @ x_1 \neq 0 \end{array} \right) = \\
 & \left(\begin{array}{l} \text{assume } x \geq 0 \\ \text{input } x_1 \\ \text{assume } 2x_1 - 1 \geq 3 \\ x_2 \leftarrow x_1 - 2 \end{array} \right); SSA_{\{x, x_1, x_2\}} \left(\begin{array}{l} \text{assume } x_2 \geq 0 \\ \text{input } x_2 \\ \text{assume } \neg 2x_2 - 1 \geq 3 \\ x_2 \leftarrow x_2 - 1 \\ @ x_2 \neq 0 \end{array} \right) =
 \end{aligned}$$

Example:

$$\begin{aligned}
 & \left(\begin{array}{l} \text{assume } x \geq 0 \\ \text{input } x_1 \\ \text{assume } 2x_1 - 1 \geq 3 \\ x_2 \leftarrow x_1 - 2 \\ \text{assume } x_2 \geq 0 \end{array} \right) ; SSA_{\{x, x_1, x_2\}} \left(\begin{array}{l} \text{input } x_2 \\ \text{assume } \neg 2x_2 - 1 \geq 3 \\ x_2 \leftarrow x_2 - 1 \\ @ x_2 \neq 0 \end{array} \right) = \\
 & \left(\begin{array}{l} \text{assume } x \geq 0 \\ \text{input } x_1 \\ \text{assume } 2x_1 - 1 \geq 3 \\ x_2 \leftarrow x_1 - 2 \\ \text{assume } x_2 \geq 0 \\ \text{input } x_3 \end{array} \right) ; SSA_{\{x, x_1, x_2, x_3\}} \left(\begin{array}{l} \text{assume } \neg 2x_3 - 1 \geq 3 \\ x_3 \leftarrow x_3 - 1 \\ @ x_3 \neq 0 \end{array} \right) = \\
 & \left(\begin{array}{l} \text{assume } x \geq 0 \\ \text{input } x_1 \\ \text{assume } 2x_1 - 1 \geq 3 \\ x_2 \leftarrow x_1 - 2 \\ \text{assume } x_2 \geq 0 \\ \text{input } x_3 \\ \text{assume } \neg 2x_3 - 1 \geq 3 \end{array} \right) ; SSA_{\{x, x_1, x_2, x_3\}} \left(\begin{array}{l} x_3 \leftarrow x_3 - 1 \\ @ x_3 \neq 0 \end{array} \right) =
 \end{aligned}$$

Example:

$$\left(\begin{array}{l} \text{assume } x \geq 0 \\ \text{input } x_1 \\ \text{assume } 2x_1 - 1 \geq 3 \\ x_2 \leftarrow x_1 - 2 \\ \text{assume } x_2 \geq 0 \\ \text{input } x_3 \\ \text{assume } \neg 2x_3 - 1 \geq 3 \\ x_4 \leftarrow x_3 - 1 \end{array} \right) ; SSA_{\{x, x_1, x_2, x_4\}} (@ x_4 \neq 0) = \left(\begin{array}{l} \text{assume } x \geq 0 \\ \text{input } x_1 \\ \text{assume } 2x_1 - 1 \geq 3 \\ x_2 \leftarrow x_1 - 2 \\ \text{assume } x_2 \geq 0 \\ \text{input } x_3 \\ \text{assume } \neg 2x_3 - 1 \geq 3 \\ x_4 \leftarrow x_3 - 1 \\ @ x_4 \neq 0 \end{array} \right)$$

First-Order Formula from SSA

Example:

assume $x_1 \geq 0$; **input** x_2 ; **assume** $2x_2 - 1 \geq 3$; $x_3 \leftarrow x_2 - 2$;
assume $x_3 \geq 0$; **input** x_4 ; **assume** $\neg 2x_4 - 1 \geq 3$; $x_5 \leftarrow x_4 - 1$;

$$x_1 \geq 0 \wedge 2x_2 - 1 \geq 3 \wedge x_3 = x_2 - 2 \wedge x_3 \geq 0 \wedge 2x_4 - 1 < 3$$

Formally:

$$F(\mathbf{assume} \ \phi) := \phi$$

$$F(\mathbf{input} \ v) := \top$$

$$F(v \leftarrow t) := v = t$$

$$F(c_1; \dots; c_n; @ \ \alpha) := \left[\bigwedge_{i \in \{1, \dots, n\}, F(c_i) \neq \top} F(c_i) \right] \Rightarrow \alpha$$

Correctness of Basic Paths

Resulting formula (*verification condition*) for a basic path P

$$VC(P) := F(SSA(P))$$

Then:

A basic path P is *correct* (as a program)

iff

$$\models VC(P)$$

Programs with Several Assertions @

$x \leftarrow 2$
 $@ x \geq 2$
 $x \leftarrow 2x$
 $@ x \geq 4$

is correct because

$x \leftarrow 2$
 $@ x \geq 2$

and

assume $x \geq 2$
 $x \leftarrow 2x$
 $@ x \geq 4$

are correct.

BUT:

$x \leftarrow 2$
 $@ 0 = 0$
 $x \leftarrow 2x$
 $@ x \geq 4$

is correct and

assume $0 = 0$
 $x \leftarrow 2x$
 $@ x \geq 4$

is not.

Programs with Several Assertions @

...
@ α_1
...
@ α_2
...
...
@ α_n

is correct **if** the following n basic paths are correct:

... **assume** α_1 ... **assume** α_{n-1}
@ α_1 ... @ α_n

To make them correct, strenghtening of $\alpha_1, \dots, \alpha_{n-1}$ might be necessary.

Strengthening of Intermediate Assertion: Example:

$x \leftarrow 2$

@ $0 = 0 \wedge x \geq 2$

$x \leftarrow 2x$

@ $x \geq 5$

Verification Conditions: Manual Proof

assume $x \leq -2$

$y \leftarrow x$

$x \leftarrow -2y + 1$

$z \leftarrow x + y$

@ $z \geq 0$

$$[x \leq -2 \wedge y = x \wedge x_1 = -2y + 1 \wedge z = x_1 + y] \Rightarrow z \geq 0$$

Assumptions: $x \leq -2$, $y = x$, $x_1 = -2y + 1$, $z = x_1 + y$

Prove $z \geq 0$

Using the assumption $z = x_1 + y$, this means to prove $x_1 + y \geq 0$

Using the assumption $x_1 = -2y + 1$, this means to prove $-2y + 1 + y \geq 0$

This means to prove $-y + 1 \geq 0$.

Using the assumption $y = x$, this means to prove $-x + 1 \geq 0$ that is $x \leq 1$.

This certainly holds under the assumption $x \leq -2$.

Counter-Examples

program:

assume $x \leq 5$

$y \leftarrow x$

$x \leftarrow -2y + 1$

$z \leftarrow x + y$

@ $z \geq 0$

bug:

$x \leftarrow 4$

$y \leftarrow 4$

$x \leftarrow -7$

$z \leftarrow -3$

error program:

assume $x \leq 5$

$y \leftarrow x$

$x \leftarrow -2y + 1$

$z \leftarrow x + y$

@ $z < 0$

verification condition (`error.cvc`):

$$[x \leq 5 \wedge y = x \wedge x_1 = -2y + 1 \wedge z = x_1 + y] \Rightarrow z \geq 0$$

counter-example:

$$\{x \mapsto 4, y \mapsto 4, x_1 \mapsto -7, z \mapsto -3\}$$

error condition:

$$x \leq -5 \wedge y = x \wedge x_1 = -2y + 1 \wedge z = x_1 + y \wedge z < 0$$

Verification Conditions and Error Conditions

$$[x \leq 5 \wedge y = x \wedge x_1 = -2y + 1 \wedge z = x_1 + y] \Rightarrow z \geq 0$$

is equivalent to

$$\neg[x \leq 5 \wedge y = x \wedge x_1 = -2y + 1 \wedge z = x_1 + y] \vee \neg z < 0$$

is equivalent to

$$\neg[x \leq 5 \wedge y = x \wedge x_1 = -2y + 1 \wedge z = x_1 + y \wedge z < 0]$$

negation of error condition, alternative form of verification condition

intuition: every execution satisfies all assertions vs.
no execution violates an assertion

to be used later

logic: a formula is invalid iff its negation is satisfiable

Summary

For proving correctness of a program without control structure we need

1. view the program as being in **SSA**, and
2. check the corresponding **verification condition**.

Such transformations of programs to logical formulas form the basis of many program analysis tools.

Literature: many variants (not all of them use SSA)

Closest approach: [Kroening and Strichman, 2016, Section 12.2]

Literature I

Aaron Bradley and Zohar Manna. *The Calculus of Computation*. Springer, 2007.

Daniel Kroening and Ofer Strichman. *Decision Procedures*. Springer, 2nd edition, 2016.